

Setting up a Data Warehouse using Amazon Redshift Serverless

SPL-BE-200-ANDEW1-1 - Version 1.0.3

© 2025 Amazon Web Services, Inc. or its affiliates. All rights reserved. This work may not be reproduced or redistributed, in whole or in part, without prior written permission from Amazon Web Services, Inc. Commercial copying, lending, or selling is prohibited. All trademarks are the property of their owners.

Note: Do not include any personal, identifying, or confidential information into the lab environment. Information entered may be visible to others.

Corrections, feedback, or other questions? Contact us at [AWS Training and Certification](#).

Lab overview

AnyCompany Events is ready for you to create an Amazon Redshift Serverless data warehouse. After you create a namespace and workgroup, you then create a schema and tables that gets loaded with TICKIT data. Finally, you create a view and query a sample of the data.

In this lab, you create an Amazon Redshift Serverless namespace and workgroup, schema, and tables. You load the tables with sample data and then query the data.

Objectives

By the end of this lab, you should be able to do the following:

- Create a data warehouse with Amazon Redshift Serverless.
- Create a schema.
- Create a table.
- Load the table with sample data.

Technical knowledge prerequisites

This lab requires the following prerequisites:

- Access to a computer with Microsoft Windows, Mac OS X, or Linux (Ubuntu, SuSE, or Red Hat)
- A modern internet browser such as Chrome or Firefox
- Basic familiarity with cloud and AWS services

Icon key

Various icons are used throughout this lab to call attention to different types of instructions and notes. The following list explains the purpose for each icon:

- **Note:** A hint, tip, or important guidance.
- **Expected output:** A sample output that you can use to verify the output of a command or edited file.

- **Command:** A command that you must run.
- **Caution:** Information of special interest or importance (not important enough to cause problems with equipment or data if you miss it, but it could result in the need to repeat certain steps).
- **Copy edit:** A time when copying a command, script, or other text to a text editor (to edit specific variables within it) might be easier than editing directly in the command line or terminal.
- **Consider:** A moment to pause to consider how you might apply a concept in your own environment or to initiate a conversation about the topic at hand.
- **Task complete:** A conclusion or summary point in the lab.

Start lab

1. To launch the lab, at the top of the page, choose **Start Lab**.

Caution: You must wait for the provisioned AWS services to be ready before you can continue.

2. To open the lab, choose **Open Console**.

You are automatically signed in to the AWS Management Console in a new web browser tab.

Warning: Do not change the **Region** unless instructed.

Common sign-in errors

Error: Choosing Start Lab has no effect

In some cases, certain pop-up or script blocker web browser extensions might prevent the **Start Lab** button from working as intended. If you experience an issue starting the lab:

- Add the lab domain name to your pop-up or script blocker's allow list or turn it off.
- Refresh the page and try again.

Lab environment

The following diagram shows the basic architecture of the lab environment:

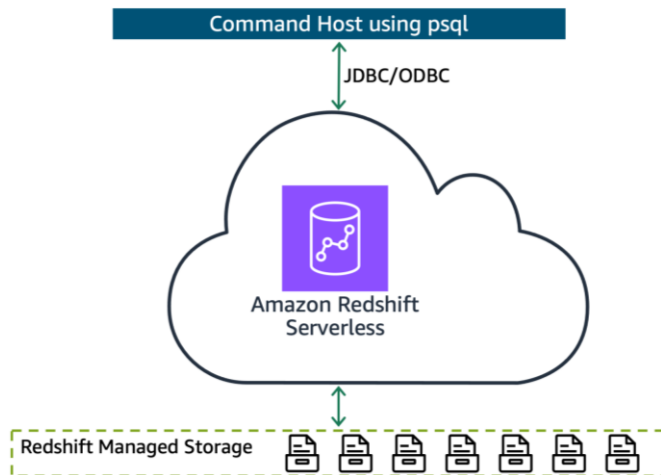


Image description: Command Host using psql communicating with Amazon Redshift Serverless. Amazon Redshift Serverless is connected to Redshift Managed Storage.

AWS services not used in this lab

AWS service capabilities used in this lab are limited to what the lab requires. Expect errors when accessing other services or performing actions beyond those provided in this lab guide.

Task 1: Create a data warehouse with Amazon Redshift Serverless

In this task, you use Amazon Redshift Serverless to set up your serverless data warehouse.

Amazon Redshift Serverless makes it simple to run and scale analytics without having to manage the instance type, instance size, lifecycle management, pausing, resuming, and so on. It automatically provisions and intelligently scales data warehouse compute capacity to deliver fast performance for even the most demanding and unpredictable workloads, and you pay only for what you use.

With Amazon Redshift Serverless, you use a console interface to reach a serverless data warehouse or APIs to build applications. Through the data warehouse, you can access your Amazon Redshift managed storage and your Amazon S3 data lake.

3. At the top of the AWS Management Console, in the search bar, search for and choose

Amazon Redshift

Caution: If the navigation menu is not displayed, choose **Menu** located near the top left section of the page to display the service menu.

4. In the left navigation pane, choose **Redshift Serverless**.

5. On the **Get started with Amazon Redshift Serverless** page, configure the following:

6. In the **Configuration** section, Select **Customize settings**.

7. In the **Namespace** section, configure:

- For **Target namespace**: Enter

lab-namespace

.

- For **Admin user credentials**: Select **Customize admin user credentials**.

- For **Admin user name**: Enter

dbadmin

.

- For **Admin password**: Select **Manually add the admin password**.
- For **Admin user password**: Copy and paste the **DBPassword** value that is listed to the left of these instructions.

8. In the **Associated IAM roles** section, configure:

- Choose **Associate IAM role**.
- Select the IAM Role that contains the string **RedshiftAccessRole** in name.
- Choose **Associate IAM roles**.

9. In the **Workgroup** section, configure:

- For **Workgroup name**: Enter

lab-workgroup

.

- For **Base capacity**, select

8

.

Note: If you select any capacity other than

8

, you will face issues while running the queries in the subsequent steps.

- For **Virtual private cloud (VPC)**: Using the dropdown menu, select the **LabVPC** value that is listed to the left of these instructions.

- For **VPC security groups**: Using the dropdown menu, select the security group with **Enable access to redshift** in the description or the **SecurityGroup** value that is listed to the left of these instructions.
- For **Subnet**: Using the dropdown menu, select **Private Subnet 1**, **Private Subnet 2**, and **Private Subnet 3** values that are listed to the left of these instructions.

10. Choose **Save configuration**.

A new window is displayed with a message *It may take a few minutes to complete. After completing the setup, you can work with your data.*

Caution: Wait for the status to display **Complete**.

11. Choose **Continue**.

Caution: If the navigation menu is not displayed, choose located near the top left section of the page to display the service menu.

12. In the left navigation pane, choose **Workgroup configuration**.

13. Choose the **lab-workgroup** link.

14. **Copy edit:** On the **lab-workgroup** page, in the **General information** section, copy the **Endpoint** value.

The endpoint should be similar to:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

```
lab-workgroup.012345678910.eu-west-2.redshift-
serverless.amazonaws.com:5439/dev
```

15. **Copy edit:** Copy the endpoint value and paste it on a notepad. Remove the **:5439/dev** portion from the end and save the remaining endpoint URL for use in the following task.

The final endpoint value should be similar to:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

```
lab-workgroup.012345678910.eu-west-2.redshift-serverless.amazonaws.com
```

Task Complete: You have successfully created an Amazon Redshift Serverless data warehouse.

Task 2: Create a schema

In this task, you create a schema in the current database.

A schema is a namespace that contains named database objects such as tables, views, and user-defined functions (UDFs). A database can contain one or multiple schemas, and each schema belongs to only one database. Two schemas can have different objects that share the same name.

Note: Amazon Redshift automatically creates a schema called `public` for every new database. When you don't specify the schema name while creating database objects, the objects go into the `public` schema.

16. **Copy edit:** Copy the **CommandHostUrlRedShift** value that is listed to the left of these instructions and then paste it into a new web browser tab to access the command host terminal.
17. **Command:** To save the value of the Redshift endpoint to an environment variable called *RedshiftEndpoint*, run the following commands on the command host terminal:
 - Replace the **REDSHIFT_ENDPOINT** placeholder value with the Endpoint value you copied in the previous task. Make sure that you have removed the **:5439/dev** portion from the end before running the command.

```
cd ~
export RedshiftEndpoint=REDSHIFT_ENDPOINT
echo $RedshiftEndpoint
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
*****

lab-workgroup.012345678910.eu-west-2.redshift-serverless.amazonaws.com
```

18. **Command:** To connect to Redshift Serverless endpoint using *psql*, run the following commands on the command host terminal:

```
echo $RedshiftDB
psql -U dbadmin -h $RedshiftEndpoint -d $RedshiftDB -p 5439
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
*****

dev
psql (15.6, server 8.0.2)
WARNING: psql major version 15, server major version 8.0.
         Some psql features might not work.
SSL connection (protocol: TLSv1.2, cipher: ECDHE-RSA-AES256-GCM-SHA384,
compression: off)
Type "help" for help.

dev=#
```

This should log you into the database and give you a prompt where you can run **SQL** commands used in this lab.

19. **Command:** To create a new schema named **US_SALES**, run the following query using the psql prompt:

The following command either creates the *US_SALES* schema for the dev database, or does nothing and returns a message if it already exists.

This sql command defines a new schema for the *dev* database.

```
create schema if not exists us_sales;
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
*****
```

```
CREATE SCHEMA
```

20. **Command:** To view the list of schemas in *dev* database, run the following query:

```
select * from pg_namespace;
```

Expected output:

```
*****
****  EXAMPLE OUTPUT  ****
*****
```

nspname	nspowner	nspacl
pg_toast	1	
pg_internal	1	
catalog_history	1	
pg_automv	1	
pg_auto_copy	1	
pg_s3	1	
pg_catalog	1	{ rdsdb=UCA/rdsdb,=U/rdsdb }
public	1	{ rdsdb=UCA/rdsdb,=UC/rdsdb }
information_schema	1	{ rdsdb=UCA/rdsdb,=U/rdsdb }
us_sales	100	

(10 rows)

Task Complete: You have successfully created a schema in dev database.

Task 3: Create a table

In this task, you create tables to hold your data.

You can also specify any column information when you create the table.

You can also create a table using the **schema_name.object_name** notation to create the table in the specified schema.

21. **Command:** To create a table in *us_sales* schema, run the following query:

```
CREATE TABLE us_sales.supplier (  
    SupplierID int,  
    City varchar (255)  
);
```

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

```
CREATE TABLE
```

22. **Command:** To insert data into a table, run the following query:

```
INSERT INTO us_sales.supplier VALUES (781, 'San Jose'), (990, 'Palo  
Alto');
```

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

```
INSERT 0 2
```

23. **Command:** To view the data inserted into a table, run the following query:

```
SELECT * from us_sales.supplier;
```

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

```
supplierid | city  
-----+-----  
          781 | San Jose  
          990 | Palo Alto  
(2 rows)
```

Task Complete: You have successfully created a table in a schema in the current database and inserted data into the table.

Task 4: Load sample data from Amazon S3

In this task, you use the **COPY** command to load data into the table.

The COPY command uses the Amazon Redshift massively parallel processing (MPP) architecture to read and load data in parallel from files in Amazon S3, from an Amazon DynamoDB table, or from text output from one or more remote hosts.

Note: The COPY command appends the new input data to any existing rows in the table. The maximum size of a single input row from any source is 4 MB.

24. **Command:** To load data into the table from Amazon S3 bucket, run the following query:

- Replace the **S3BUCKETPATH** placeholder value with the **DataSetS3Uri** value listed to the left of these instructions.(Be sure to keep the single quote marks.)
- Replace the **INSERT_REDSHIFT_ROLE** placeholder value with the **RedshiftAccessRole** value listed to the left of these instructions. (Be sure to keep the single quote marks.)
- Replace the **REGION** placeholder value with the **Region** value listed to the left of these instructions. (Be sure to keep the single quote marks.)

```
COPY us_sales.supplier
FROM 'S3BUCKETPATH/supplier.csv'
iam_role 'INSERT_REDSHIFT_ROLE'
region 'REGION' csv;
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

```
INFO: Load into table 'supplier' completed, 5 record(s) loaded
successfully.
COPY
```

25. **Command:** To query the **supplier** table, run the following query:

```
SELECT * FROM us_sales.supplier;
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

```
supplierid |      city
-----+-----
          781 | San Jose
          990 | Palo Alto
          220 | San Diego
          437 | Oakland
          100 | New Haven
          332 | Kansas City
          547 | Fort Smith
(7 rows)
```

Task Complete: You have successfully loaded sample data from Amazon S3 by using the COPY command.

Task 5: Ingest and query TICKIT data (Optional)

AnyCompany Events wants to build database application that helps analysts track sales activity from TICKIT website where users buy and sell tickets online for sporting events, shows, and concerts.

In this task, you load the TICKIT dataset from Amazon S3 to Redshift Serverless database and build queries to help analysts to use this information to provide incentives to buyers and sellers who frequent the site, to attract new users, and to drive advertising and promotions.

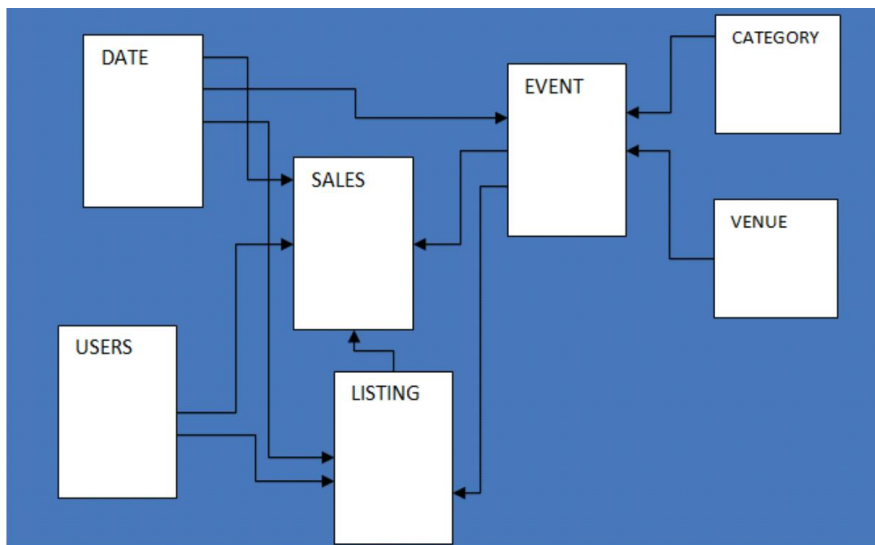


Image description: The preceding diagram depicts the Entity Relationship diagram for TICKIT database which consists of seven tables: two fact tables and five dimensions.

Task 5.1: CREATE TABLES

26. **Command:** To create tables in the dev database, individually copy and run the following create table queries:

```
create table us_sales.users(  
    userid integer not null distkey sortkey,  
    username char(8),  
    firstname varchar(30),  
    lastname varchar(30),  
    city varchar(30),  
    state char(2),  
    email varchar(100),  
    phone char(14),  
    likesports boolean,  
    liketheatre boolean,  
    likeconcerts boolean,  
    likejazz boolean,  
    likeclassical boolean,
```

```

likeopera boolean,
likerock boolean,
likevegas boolean,
likebroadway boolean,
likemusicals boolean);

```

Expected output:

```

*****
**** EXAMPLE OUTPUT ****
*****

```

CREATE TABLE

```

create table us_sales.venue(
    venueid smallint not null distkey sortkey,
    venueName varchar(100),
    venueCity varchar(30),
    venueState char(2),
    venueSeats integer);

```

Expected output:

```

*****
**** EXAMPLE OUTPUT ****
*****

```

CREATE TABLE

```

create table us_sales.category(
    catid smallint not null distkey sortkey,
    catgroup varchar(10),
    catname varchar(10),
    catdesc varchar(50));

```

Expected output:

```

*****
**** EXAMPLE OUTPUT ****
*****

```

CREATE TABLE

```

create table us_sales.date(
    dateid smallint not null distkey sortkey,
    caldate date not null,
    day character(3) not null,
    week smallint not null,
    month character(5) not null,
    qtr character(5) not null,
    year smallint not null,
    holiday boolean default('N'));

```

Expected output:

```

*****
**** EXAMPLE OUTPUT ****
*****

```

CREATE TABLE

```
create table us_sales.event(  
    eventid integer not null distkey,  
    venueid smallint not null,  
    catid smallint not null,  
    dateid smallint not null sortkey,  
    eventname varchar(200),  
    starttime timestamp);
```

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

CREATE TABLE

```
create table us_sales.listing(  
    listid integer not null distkey,  
    sellerid integer not null,  
    eventid integer not null,  
    dateid smallint not null sortkey,  
    numtickets smallint not null,  
    priceperticket decimal(8,2),  
    totalprice decimal(8,2),  
    listtime timestamp);
```

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

CREATE TABLE

```
create table us_sales.sales(  
    salesid integer not null,  
    listid integer not null distkey,  
    sellerid integer not null,  
    buyerid integer not null,  
    eventid integer not null,  
    dateid smallint not null sortkey,  
    qtysold smallint not null,  
    pricepaid decimal(8,2),  
    commission decimal(8,2),  
    saletime timestamp);
```

Expected output:

```
*****  
**** EXAMPLE OUTPUT ****  
*****
```

CREATE TABLE

Task 5.2: Load data set from Amazon S3 by using the COPY command

27. **Command:** To load TICKIT dataset from the Amazon S3 bucket, individually copy and run the following queries:

- Replace the **S3BUCKETPATH** placeholder value with the **DataSetS3Uri** value listed to the left of these instructions. (Be sure to keep the single quote marks.)
- Replace the **INSERT_REDSHIFT_ROLE** placeholder value with the **RedshiftAccessRole** value listed to the left of these instructions. (Be sure to keep the single quote marks.)
- Replace the **REGION** placeholder value with the **Region** value listed to the left of these instructions. (Be sure to keep the single quote marks.)

```
copy us_sales.users from 'S3BUCKETPATH/allusers.csv' iam_role
'INSERT_REDSHIFT_ROLE' region 'REGION' csv;
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

```
INFO: Load into table 'users' completed, 49990 record(s) loaded
successfully.
COPY
```

```
copy us_sales.venue from 'S3BUCKETPATH/venue_pipe.txt' iam_role
'INSERT_REDSHIFT_ROLE' delimiter '|' region 'REGION';
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

```
INFO: Load into table 'venue' completed, 202 record(s) loaded
successfully.
COPY
```

```
copy us_sales.category from 'S3BUCKETPATH/category_pipe.txt' iam_role
'INSERT_REDSHIFT_ROLE' delimiter '|' region 'REGION';
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

```
INFO: Load into table 'category' completed, 11 record(s) loaded
successfully.
COPY
```

```
copy us_sales.date from 'S3BUCKETPATH/date2008.csv' iam_role
'INSERT_REDSHIFT_ROLE' region 'REGION' csv;
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

INFO: Load into table 'date' completed, 365 record(s) loaded successfully.
COPY

```
copy us_sales.event from 'S3BUCKETPATH/allevnts_pipe.txt' iam_role
'INSERT_REDSHIFT_ROLE' delimiter '|' timeformat 'YYYY-MM-DD HH:MI:SS'
region 'REGION';
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

INFO: Load into table 'event' completed, 8798 record(s) loaded successfully.
COPY

```
copy us_sales.listing from 'S3BUCKETPATH/listings_pipe.txt' iam_role
'INSERT_REDSHIFT_ROLE' delimiter '|' region 'REGION';
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

INFO: Load into table 'listing' completed, 192497 record(s) loaded successfully.
COPY

```
copy us_sales.sales from 'S3BUCKETPATH/sales_tab.txt' iam_role
'INSERT_REDSHIFT_ROLE' delimiter '\t' timeformat 'MM/DD/YYYY HH:MI:SS'
region 'REGION';
```

Expected output:

```
*****
**** EXAMPLE OUTPUT ****
*****
```

INFO: Load into table 'sales' completed, 172456 record(s) loaded successfully.
COPY

Consider: AnyCompany Events is planning a new concert series. They want to know what are the top five venues that most events happened on a given calendar date to choose the best venue for their next concert.

28. **Command:** To find top five venues that most events happened on a given calendar date, run the following query:

```
SELECT caldate, venue, count(event.eventid) total_events
```

```

FROM us_sales.sales as sales,
     us_sales.event as event,
     us_sales.venue as venue,
     us_sales.date as date
WHERE sales.eventid=event.eventid
AND event.dateid=date.dateid
AND event.venueid=venue.venueid
GROUP BY caldate,venueid
ORDER BY total_events DESC
LIMIT 5;

```

Expected output:

```

*****
****  EXAMPLE OUTPUT  ****
*****

```

caldate	venueid	total_events
2008-05-08	Charles Playhouse	108
2008-11-17	FedExForum	86
2008-11-24	Belasco Theatre	83
2008-03-10	Eugene O'Neill Theatre	79
2008-04-07	Madison Square Garden	71

(5 rows)

Consider: You are encouraged to write your own queries using the Ticket data set and help the analysts find the information they require.

Task Complete: You have successfully loaded the TICKIT dataset from Amazon S3 to Redshift Serverless database and built queries to help analysts to use this information to provide incentives to buyers and sellers who frequent the site, to attract new users, and to drive advertising and promotions.

Conclusion

You have successfully done the following:

- Created a data warehouse with Amazon Redshift Serverless.
- Created a schema.
- Created a table.
- Loaded the table with sample data.

End lab

Follow these steps to close the console and end your lab.

29. Return to the **AWS Management Console**.

30. At the upper-right corner of the page, choose **AWS Labs User**, and then choose **Sign out**.

31. Choose **End Lab** and then confirm that you want to end your lab.

For more information about AWS Training and Certification, see <https://aws.amazon.com/training/>.

Your feedback is welcome and appreciated.

If you would like to share any feedback, suggestions, or corrections, please provide the details in our [AWS Training and Certification Contact Form](#).